Java w IntelliJ IDEA - podejmowanie decyzji

BLOK 3 - porównania, warunki, instrukcje if i switch

1. Dlaczego program potrzebuje instrukcji sterujących?

Dotychczasowe przykłady wykonywały kolejne instrukcje od góry do dołu. W praktyce program często musi wybrać sposób działania. Jeżeli użytkownik ma co najmniej 18 lat, wolno mu zobaczyć określony komunikat; jeśli nie, program powinien wyświetlić inną wiadomość. Taką decyzję opisuje instrukcja warunkowa. Warunek nie jest pytaniem zadawanym człowiekowi, lecz wyrażeniem, którego wynikiem musi być `true` albo `false`.

```
int age = 19;
if (age >= 18) {
    System.out.println("Osoba pełnoletnia");
}
System.out.println("Koniec sprawdzania");
```

```
Wynik:
Osoba pełnoletnia
Koniec sprawdzania
```

Program najpierw odczytuje `age`, potem sprawdza wyrażenie `age >= 18`. Ponieważ 19 jest większe lub równe 18, wynik warunku to `true` i Java wykonuje instrukcję w nawiasach klamrowych. Ostatnia instrukcja znajduje się poza blokiem `if`, dlatego jest wykonywana niezależnie od wieku. Gdy zmienimy wiek na 17, pierwszy komunikat zniknie, ale napis „Koniec sprawdzania” pozostanie.

2. Operatory porównania - co zwracają?

Do porównywania wartości używamy operatorów == (równe), `!=` (różne), > (większe), < (mniejsze), >= (większe lub równe) i <= (mniejsze lub równe). Każdy z tych operatorów tworzy wyrażenie logiczne. Na przykład `5 > 3` daje `true`, a `5 == 3` daje `false`.

```
int score = 50;
System.out.println(score > 50);
System.out.println(score >= 50);
System.out.println(score != 0);
```

```
Wynik:
false
true
true
```

Pierwsze porównanie jest fałszywe, ponieważ 50 nie jest **większe** od 50. Drugie jest prawdziwe, gdyż znak >= uwzględnia również równość. To rozróżnienie jest szczególnie istotne przy wartościach granicznych. Jeżeli promocja obowiązuje dla zakupów od 100 zł włącznie, warunek powinien używać `>= 100`, a nie `> 100`.

3. Przypisanie = i sprawdzenie równości ==

Pojedynczy znak `=` oznacza przypisanie wartości do zmiennej. Podwójny `==` oznacza porównanie. Pomylenie tych zapisów jest jednym z częstszych błędów na początku nauki. W Javie instrukcja `if (age = 18)` nie jest poprawna, jeżeli `age` ma typ `int`, ponieważ przypisanie liczby nie daje wyniku typu `boolean`.

```
int a = 10;
int b = 10;
System.out.println(a == b); // porównanie
b = 11;                     // przypisanie
System.out.println(a == b);
```

```
Wynik:
true
false
```

Po przypisaniu `b = 11` zmienia się zawartość zmiennej `b`. Porównanie nie zmienia żadnej zmiennej: tylko sprawdza, czy dwie wartości są równe.

4. Instrukcja if oraz blok nawiasów klamrowych

Słowo `if` oznacza „jeżeli”. Po nim w nawiasach okrągłych umieszczamy wyrażenie logiczne. Gdy wynik to `true`, Java wykonuje instrukcje umieszczone w bloku `{ ... }`. Gdy wynik to `false`, cały blok jest pomijany. Samo wcięcie kodu nie decyduje o tym, które instrukcje należą do bloku - robią to nawiasy klamrowe.

```
int temperature = -2;
if (temperature < 0) {
    System.out.println("Uwaga na oblodzenie");
    System.out.println("Załóż ciepłe ubranie");
}
System.out.println("Sprawdzenie zakończone");
```

Oba komunikaty w bloku `if` pojawią się, gdy temperatura jest ujemna. Gdy zmienimy temperaturę na 3, żaden z nich się nie pojawi, lecz ostatni komunikat pozostanie. Warto zawsze prześledzić, gdzie zaczyna się i kończy para klamer. Pominięcie ich w prostym `if` jest dozwolone przy jednej instrukcji, ale często prowadzi do trudnych do zauważenia błędów podczas rozbudowy programu.

5. if-else - wybór jednej z dwóch możliwości

Instrukcja `else` określa, co ma wykonać program, gdy warunek `if` jest fałszywy. W jednym przejściu przez taki fragment programu wykona się dokładnie jedna z dwóch gałęzi. Nie może być jednocześnie spełnione „warunek jest prawdziwy” i „warunek jest fałszywy”.

```
int points = 46;
if (points >= 50) {
    System.out.println("Zaliczone");
} else {
    System.out.println("Nie zaliczono");
}
```

```
Wynik:
Nie zaliczono
```

Program sprawdza, czy 46 jest co najmniej równe 50. Odpowiedź jest fałszywa, zatem wykonywany jest kod po `else`. Zwróć uwagę, że `else` nie ma osobnego warunku. Jego znaczenie to „we wszystkich pozostałych przypadkach”.

6. else-if - więcej niż dwie możliwości

W wielu zadaniach mamy kilka kategorii, na przykład ocenę zależną od liczby punktów. Wtedy można użyć `if`, kilku `else if` i opcjonalnego `else` na końcu. Warunki są sprawdzane kolejno, aż zostanie znaleziony pierwszy prawdziwy. Późniejsze gałęzie nie będą już wykonywane.

```
int score = 82;
if (score >= 90) {
    System.out.println("Bardzo dobry");
} else if (score >= 70) {
    System.out.println("Dobry");
} else if (score >= 50) {
    System.out.println("Dostateczny");
} else {
    System.out.println("Niezaliczony");
}
```

```
Wynik:
Dobry
```

Najpierw pytamy, czy 82 jest co najmniej 90 - nie. Następnie sprawdzamy próg 70 - tak. Java wykonuje komunikat „Dobry” i nie sprawdza już niższych progów. Dlatego kolejność progów ma znaczenie. Jeżeli umieścisz `score >= 50` jako pierwszy warunek, wynik 82 trafi do kategorii dostatecznej i program będzie logicznie niepoprawny.

7. Operatory logiczne &&, || oraz !

Kiedy decyzja zależy od więcej niż jednego warunku, używamy operatorów logicznych. `&&` oznacza logiczne „i”: obie strony muszą być prawdziwe. `||` oznacza „lub”: wystarczy co najmniej jedna prawdziwa strona. `!` zaprzecza wartości logicznej, czyli zmienia `true` na `false` i odwrotnie.

```
boolean hasTicket = true;
boolean isAdult = false;
System.out.println(hasTicket && isAdult);
System.out.println(hasTicket || isAdult);
System.out.println(!isAdult);
```

```
Wynik:
false
true
true
```

W pierwszym przypadku użytkownik nie spełnia obu warunków, bo `isAdult` jest fałszywe. W drugim wystarczy ważny bilet, więc wynik jest prawdziwy. W trzecim `!false` daje `true`. Warunki należy zapisywać na podstawie treści zadania: zdanie „ma bilet i jest pełnoletni” oznacza `&&`, a nie `||`.

Operator	**Kiedy wynik to true?**	**Przykład**

&&	**gdy obie strony są prawdziwe**	`true && false` **daje** `false`
`		`
!	gdy negowany warunek jest fałszywy	`!false` daje `true`

8. Jak działa skrócone obliczanie warunków?

Java przy operatorze `&&` nie musi obliczać prawej strony, jeśli lewa jest już fałszywa. Przy `||` nie musi obliczać prawej strony, jeśli lewa jest prawdziwa. Jest to **short-circuit**, czyli obliczanie skrócone. Możemy wykorzystać tę zasadę, aby nie wykonać operacji, która byłaby niebezpieczna dla pewnych danych.

```
int divisor = 0;
if (divisor != 0 && 10 / divisor > 2) {
    System.out.println("Duży iloraz");
} else {
    System.out.println("Nie można spełnić warunku");
}
```

```
Wynik:
Nie można spełnić warunku
```

Gdy `divisor` wynosi 0, pierwsza część warunku daje `false`. Druga część zawierająca dzielenie nie zostaje wykonana. Gdybyśmy użyli nieodpowiedniego zapisu i program próbował obliczyć `10 / 0`, otrzymalibyśmy wyjątek. Skrócone obliczanie jest zatem ważne nie tylko dla szybkości, ale i dla poprawności.

9. Porównywanie napisów String

Dla liczb zapis `==` porównuje wartości. Dla obiektów, na przykład typu `String`, `==` sprawdza, czy dwie referencje wskazują na ten sam obiekt, a nie po prostu czy napisy mają te same litery. Aby porównać treść napisów, używamy `equals`.

```
String first = new String("java");
String second = new String("java");
System.out.println(first == second);
System.out.println(first.equals(second));
```

```
Wynik:
false
true
```

Mimo że oba napisy zawierają te same litery, w przykładzie tworzone są dwa różne obiekty. `equals` bada ich treść, dlatego daje `true`. Przy porównywaniu wartości, która może być `null`, wygodny zapis to `"java".equals(input)`, bo wywołanie metody na stałym, niepustym napisie nie zgłosi `NullPointerException`.

10. switch - wybór działania na podstawie jednej wartości

`switch` przydaje się, gdy decyzję podejmujemy na podstawie jednej wartości, na przykład numeru dnia tygodnia lub wybranej opcji. W tradycyjnej postaci zapisujemy przypadki `case`, a instrukcja `break` pozwala zakończyć obsługę wybranego przypadku. Bez `break` w klasycznej składni program może przejść do kolejnej etykiety.

```
int option = 2;
switch (option) {
    case 1:
        System.out.println("Dodawanie");
        break;
    case 2:
        System.out.println("Odejmowanie");
        break;
    default:
        System.out.println("Nieznana opcja");
}
```

```
Wynik:
Odejmowanie
```

Program szuka `case 2`, wypisuje tekst i kończy `switch` po napotkaniu `break`. `default` jest odpowiednikiem przypadku, w którym żadna wymieniona etykieta nie pasuje. Nie zastępuj `switch` wszystkimi możliwymi instrukcjami `if`; wybierz tę formę, która lepiej odpowiada problemowi. W Javie istnieje też nowsza składnia ze strzałką `case 2 ->`, która nie wymaga `break` w analogicznym zastosowaniu.

11. Operator warunkowy ?: - skrócona decyzja

Operator `?:` umożliwia wybór jednej z dwóch wartości na podstawie warunku. Zapis ma trzy części: warunek, wynik dla `true` i wynik dla `false`. Nadaje się do prostych decyzji, które nie wymagają wielu dodatkowych instrukcji.

```
int age = 16;
String status = age >= 18 ? "pełnoletni" : "niepełnoletni";
System.out.println(status);
```

```
Wynik:
niepełnoletni
```

Czytamy to następująco: „jeśli wiek jest co najmniej 18, użyj pierwszego napisu, w przeciwnym razie drugiego”. Operator zwraca wybraną wartość, którą zapisujemy do zmiennej `status`. Jeżeli warunek i wyniki są zbyt skomplikowane, czytelniejsza będzie zwykła instrukcja `if-else`.

Przykład 1. Example01Comparisons

Porównujemy liczby. Każdy operator tworzy wynik logiczny `true` albo `false`.

```
package block3;

public class Example01Comparisons {
    public static void main(String[] args) {
        int age = 20;
        System.out.println(age >= 18);
        System.out.println(age == 20);
        System.out.println(age != 21);
        System.out.println(age < 30);
    }
}
```

Wynik:

```
true
true
true
true
```

- Zapis `int age = 20;` tworzy zmienną `age` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Instrukcja `System.out.println(age >= 18);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println(age == 20);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println(age != 21);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println(age < 30);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 2. Example02Logical

Łączymy warunki dotyczące wieku, biletu i zakazu wejścia. Oddzielnie obliczamy warunek uprawniający do zniżki.

```
package block3;

public class Example02Logical {
    public static void main(String[] args) {
        int age = 22;
        boolean hasTicket = true;
        boolean banned = false;

        boolean canEnter = age >= 18 && hasTicket && !banned;
        boolean discount = age < 18 || age >= 65;

        System.out.println("Wejście: " + canEnter);
        System.out.println("Zniżka: " + discount);
    }
}
```

Wynik:

```
Wejście: true
Zniżka: false
```

- Zapis `int age = 22;` tworzy zmienną `age` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `boolean hasTicket = true;` tworzy zmienną `hasTicket` typu `boolean` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `boolean banned = false;` tworzy zmienną `banned` typu `boolean` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `boolean canEnter = age >= 18 && hasTicket && !banned;` tworzy zmienną `canEnter` typu `boolean` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `boolean discount = age < 18 || age >= 65;` tworzy zmienną `discount` typu `boolean` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Instrukcja `System.out.println("Wejście: " + canEnter);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 3. Example03If

Wykonujemy blok instrukcji tylko wtedy, gdy podany warunek jest prawdziwy.

```
package block3;

public class Example03If {
    public static void main(String[] args) {
        int temperature = -2;

        if (temperature < 0) {
            System.out.println("Uwaga: mróz");
        }

        System.out.println("Koniec programu");
    }
}
```

Wynik:

```
Uwaga: mróz
Koniec programu
```

- Zapis `int temperature = -2;` tworzy zmienną `temperature` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- W zapisie `if (temperature < 0) {` program podejmuje decyzję na podstawie wartości logicznej. Instrukcje w bloku są wykonywane tylko wtedy, gdy wynik warunku jest `true`.
- Instrukcja `System.out.println("Uwaga: mróz");` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println("Koniec programu");` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 4. Example04IfElseChain

Przypisujemy wynikowi punktowemu jedną z kategorii. O wyniku decyduje pierwszy prawdziwy warunek.

```
package block3;

public class Example04IfElseChain {
    public static void main(String[] args) {
        int score = 82;

        if (score >= 90) {
            System.out.println("Ocena 5");
        } else if (score >= 75) {
            System.out.println("Ocena 4");
        } else if (score >= 60) {
            System.out.println("Ocena 3");
        } else {
            System.out.println("Brak zaliczenia");
        }
    }
}
```

Wynik:

```
Ocena 4
```

- Zapis `int score = 82;` tworzy zmienną `score` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- W zapisie `if (score >= 90) {` program podejmuje decyzję na podstawie wartości logicznej. Instrukcje w bloku są wykonywane tylko wtedy, gdy wynik warunku jest `true`.
- Instrukcja `System.out.println("Ocena 5");` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println("Ocena 4");` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println("Ocena 3");` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

- Instrukcja `System.out.println("Brak zaliczenia");` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 5. Example05Switch

Wybieramy komunikat na podstawie jednej wartości przy użyciu instrukcji `switch`.

```
package block3;

public class Example05Switch {
    public static void main(String[] args) {
        int day = 3;

        String name = switch (day) {
            case 1 -> "poniedziałek";
            case 2 -> "wtorek";
            case 3 -> "środa";
            case 4 -> "czwartek";
            case 5 -> "piątek";
            case 6, 7 -> "weekend";
            default -> "niepoprawny numer";
        };

        System.out.println(name);
    }
}
```

Wynik:

```
środa
```

- Zapis `int day = 3;` tworzy zmienną `day` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Fragment `case 1 -> "poniedziałek";` oznacza jeden z możliwych przypadków rozpatrywanych przez `switch`.
- Fragment `case 2 -> "wtorek";` oznacza jeden z możliwych przypadków rozpatrywanych przez `switch`.
- Fragment `case 3 -> "środa";` oznacza jeden z możliwych przypadków rozpatrywanych przez `switch`.
- Fragment `case 4 -> "czwartek";` oznacza jeden z możliwych przypadków rozpatrywanych przez `switch`.
- Fragment `case 5 -> "piątek";` oznacza jeden z możliwych przypadków rozpatrywanych przez `switch`.

Przykład 6. Example06Ternary

Używamy operatora `?:` do wyboru jednej z dwóch wartości bez rozbudowanego bloku `if`.

```
package block3;

public class Example06Ternary {
    public static void main(String[] args) {
        int age = 17;
        String status = age >= 18 ? "pełnoletni" : "niepełnoletni";
        System.out.println(status);
    }
}
```

Wynik:

```
niepełnoletni
```

- Zapis `int age = 17;` tworzy zmienną `age` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `String status = age >= 18 ? "pełnoletni" : "niepełnoletni";` tworzy zmienną `status` typu `String` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Instrukcja `System.out.println(status);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Zadanie 7. LoginCheck

Sprawdzamy, czy kilka warunków dostępu jest spełnionych jednocześnie.

Polecenie: Dostęp jest dozwolony, jeśli konto jest aktywne, użytkownik nie jest zablokowany i podał poprawne hasło. Zapisz jeden warunek boolean, a potem if/else.

```
package block3;

public class LoginCheck {
    public static void main(String[] args) {
        boolean active = true;
        boolean blocked = false;
        boolean correctPassword = true;
        // TODO
    }
}
```

- Zapis `boolean active = true;` tworzy zmienną `active` typu `boolean` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `boolean blocked = false;` tworzy zmienną `blocked` typu `boolean` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `boolean correctPassword = true;` tworzy zmienną `correctPassword` typu `boolean` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.

Zadanie 8. LeapYear

Sprawdzamy regułę roku przestępnego, która łączy kilka porównań i operatorów logicznych.

Polecenie: Rok jest przestępny, gdy: • dzieli się przez 400, LUB • dzieli się przez 4 i jednocześnie nie dzieli się przez 100.

```
package block3;

public class LeapYear {
    public static void main(String[] args) {
        int year = 2028;
        // TODO
    }
}
```

- Zapis `int year = 2028;` tworzy zmienną `year` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.

Zadanie 9. TicketPrice

W klasie `TicketPrice` należy samodzielnie uzupełnić algorytm zgodnie z komentarzem w pliku. Poniżej widoczny jest kod początkowy, do którego wpisujemy rozwiązanie.

Polecenie: Cena bazowa 30.0. Dzieci < 12: 50% ceny, seniorzy >= 65: 70% ceny, pozostali pełna cena. Użyj operatora ?: tylko tam, gdzie kod pozostaje czytelny.

```
package block3;

public class TicketPrice {
    public static void main(String[] args) {
        int age = 70;
        double basePrice = 30.0;
        // TODO
    }
}
```

- Zapis `int age = 70;` tworzy zmienną `age` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `double basePrice = 30.0;` tworzy zmienną `basePrice` typu `double` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.

12. Zadania - decyzje i porównania

Zadanie 1. Dla wieku 17, 18 i 19 przetestuj warunek pełnoletniości. Zapisz przewidywany wynik przed uruchomieniem. Wyjaśnij, dlaczego porównanie `>= 18` jest odpowiedniejsze niż `> 18`.

Zadanie 2. Utwórz zmienne `boolean hasTicket` i `boolean isAdult`. Napisz warunek, który przyzna wstęp tylko wtedy, gdy obie wartości są prawdziwe. Przetestuj wszystkie cztery kombinacje wartości.

Zadanie 3. Dla liczby punktów od 0 do 100 zapisz cztery kategorie: od 90, od 70, od 50 oraz poniżej 50. Sprawdź wartości graniczne 49, 50, 69, 70, 89, 90. Nie zmieniaj kodu `println`; zmieniaj tylko liczbę punktów.

Zadanie 4. Napisz `switch` dla numeru dnia 1-7. Dla liczb 1-5 wypisz „Dzień roboczy”, dla 6-7 „Weekend”, dla wszystkich pozostałych „Nieprawidłowy numer”. Możesz pogrupować przypadki.

Zadanie 5. W klasie `LeapYear` uzupełnij i przetestuj regułę: rok jest przestępny, jeżeli dzieli się przez 400, albo dzieli się przez 4 i nie dzieli się przez 100. Sprawdź lata 1900, 2000, 2024 i 2025.